

learned (e.g. AlphaGo [29]). We propose a third alternative, by using the LM to deliberately reason about states. When applicable, such a deliberate heuristic can be more flexible than programmed rules, and more sample-efficient than learned models. Similar to the thought generator, we consider two strategies to evaluate states either independently or together:

- (a) **Value** each state independently: $V(p_\theta, S)(s) \sim p_\theta^{value}(v|s) \forall s \in S$, where a value prompt reasons about the state s to generate a scalar value v (e.g. 1-10) or a classification (e.g. sure/likely/impossible) that could be heuristically turned into a value. The basis of such evaluative reasoning can vary across problems and thought steps. In this work, we explore evaluation via few *lookahead* simulations (e.g. quickly confirm that 5, 5, 14 can reach 24 via $5 + 5 + 14$, or “hot.l” can mean “inn” via filling “e” in “_”) plus commonsense (e.g. 1 2 3 are too small to reach 24, or no word can start with “tzxc”). While the former might promote “good” states, the latter could help eliminate “bad” states. Such valuations do not need to be perfect, and only need to be approximately helpful for decision making.
- (b) **Vote** across states: $V(p_\theta, S)(s) = \mathbb{1}[s = s^*]$, where a “good” state $s^* \sim p_\theta^{vote}(s^*|S)$ is voted out based on deliberately comparing different states in S in a vote prompt. When problem success is harder to directly value (e.g. passage coherency), it is natural to instead compare different partial solutions and vote for the most promising one. This is similar in spirit to a “step-wise” self-consistency strategy, i.e. cast “which state to explore” as a multi-choice QA, and use LM samples to vote for it.

For both strategies, we could prompt the LM multiple times to aggregate the value or vote results to trade time/resource/cost for more faithful/robust heuristics.

Algorithm 1 ToT-BFS($x, p_\theta, G, k, V, T, b$)	Algorithm 2 ToT-DFS($s, t, p_\theta, G, k, V, T, v_{th}$)
Require: Input x , LM p_θ , thought generator $G()$ & size limit k , states evaluator $V()$, step limit T , breadth limit b . $S_0 \leftarrow \{x\}$ for $t = 1, \dots, T$ do $S'_t \leftarrow \{[s, z] \mid s \in S_{t-1}, z_t \in G(p_\theta, s, k)\}$ $V_t \leftarrow V(p_\theta, S'_t)$ $S_t \leftarrow \arg \max_{S \subset S'_t, S =b} \sum_{s \in S} V_t(s)$ end for return $G(p_\theta, \arg \max_{s \in S_T} V_T(s), 1)$	Require: Current state s , step t , LM p_θ , thought generator $G()$ and size limit k , states evaluator $V()$, step limit T , threshold v_{th} if $t > T$ then record output $G(p_\theta, s, 1)$ end if for $s' \in G(p_\theta, s, k)$ do $\triangleright$ sorted candidates if $V(p_\theta, \{s'\})(s) > v_{thres}$ then $\triangleright$ pruning DFS($s', t + 1$) end if end for

4. Search algorithm. Finally, within the ToT framework, one can plug and play different search algorithms depending on the tree structure. We explore two relatively simple search algorithms and leave more advanced ones (e.g. A* [11], MCTS [2]) for future work:

- (a) **Breadth-first search (BFS)** (Algorithm 1) maintains a set of the b most promising states per step. This is used for Game of 24 and Creative Writing where the tree depth is limit ($T \leq 3$), and initial thought steps can be evaluated and pruned to a small set ($b \leq 5$).
- (b) **Depth-first search (DFS)** (Algorithm 2) explores the most promising state first, until the final output is reached ($t > T$), or the state evaluator deems it impossible to solve the problem from the current s ($V(p_\theta, \{s'\})(s) \leq v_{th}$ for a value threshold v_{th}). In the latter case, the subtree from s is *pruned* to trade exploration for exploitation. In both cases, DFS *backtracks* to the parent state of s to continue exploration.

Conceptually, ToT has several benefits as a method for general problem-solving with LMs: (1) *Generality*. IO, CoT, CoT-SC, and self-refinement can be seen as special cases of ToT (i.e. trees of limited depth and breadth; Figure 1). (2) *Modularity*. The base LM, as well as the thought decomposition, generation, evaluation, and search procedures can all be varied independently. (3) *Adaptability*. Different problem properties, LM capabilities, and resource constraints can be accommodated. (4) *Convenience*. No extra training is needed, just a pre-trained LM is sufficient. The next section will show how these conceptual benefits translate to strong empirical performance in different problems.

4 Experiments

We propose three tasks that are hard even when sampling from the state-of-the-art language model, GPT-4 [23], using standard IO prompting or chain-of-thought (CoT) prompting. We show how

	Game of 24	Creative Writing	5x5 Crosswords
Input	4 numbers (4 9 10 13)	4 random sentences	10 clues (h1. presented;..)
Output	An equation to reach 24 (13-9)*(10-4)=24	A passage of 4 paragraphs ending in the 4 sentences	5x5 letters: SHOWN ; WIRRA ; AVAIL ; ...
Thoughts	3 intermediate equations (13-9=4 (left 4,4,10); 10-4=6 (left 4,6); 4*6=24)	A short writing plan (1. Introduce a book that connects...)	Words to fill in for clues: (h1. shown; v5. naled; ...)
#ToT steps	3	1	5-10 (variable)

Table 1: Task overview. Input, output, thought examples are in blue.

deliberate search in trees of thoughts (ToT) produces better results, and more importantly, interesting and promising new ways to use language models to solve problems requiring search or planning. Unless otherwise stated, we perform experiments using a Chat Completion mode GPT-4¹ with a sampling temperature of 0.7.

4.1 Game of 24

Game of 24 is a mathematical reasoning challenge, where the goal is to use 4 numbers and basic arithmetic operations (+-*/) to obtain 24. For example, given input “4 9 10 13”, a solution output could be “(10 - 4) * (13 - 9) = 24”.

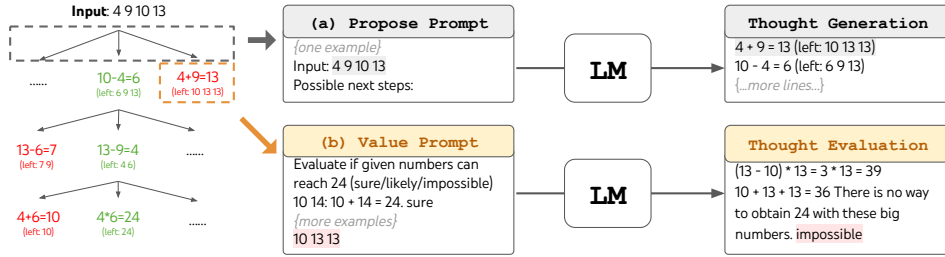


Figure 2: ToT in a game of 24. The LM is prompted for (a) thought generation and (b) valuation.

Task Setup. We scrape data from 4nums.com, which has 1,362 games that are sorted from easy to hard by human solving time, and use a subset of relatively hard games indexed 901-1,000 for testing. For each task, we consider the output as success if it is a valid equation that equals 24 and uses the input numbers each exactly once. We report the success rate across 100 games as the metric.

Baselines. We use a standard input-output (IO) prompt with 5 in-context examples. For chain-of-thought (CoT) prompting, we augment each input-output pair with 3 intermediate equations, each operating on two remaining numbers. For example, given input “4 9 10 13”, the thoughts could be “13 - 9 = 4 (left: 4 4 10); 10 - 4 = 6 (left: 4 6); 4 * 6 = 24 (left: 24)”. For each game, we sample IO and CoT prompting for 100 times for average performance. We also consider a CoT self-consistency baseline, which takes the majority output from 100 CoT samples, and an iterative-refine approach on top of an IO sample for at most 10 iterations. At each iteration, the LM is conditioned on all previous history to “reflect on your mistakes and generate a refined answer” if the output is incorrect. Note that it uses groundtruth feedback signals about equation correctness.

ToT Setup. To frame Game of 24 into ToT, it is natural to decompose the thoughts into 3 steps, each an intermediate equation. As shown in Figure 2(a), at each tree node, we extract the remaining numbers and prompt the LM to propose some possible next steps. The same “propose prompt” is used for all 3 thought steps, though it only has one example with 4 input numbers. We perform a breadth-first search (BFS) in ToT, where at each step we keep the best $b = 5$ candidates. To perform deliberate BFS in ToT, as shown in Figure 2(b), we prompt LM to evaluate each thought candidate as “sure/maybe/impossible” with regard to reaching 24. The aim is to promote correct partial solutions that can be verdicted within few lookahead trials, and eliminate impossible partial solutions based on “too big/small” commonsense, and keep the rest “maybe”. We sample values 3 times for each thought.

¹Experiments were done between May 5-16, 2023.

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC ($k=100$)	9.0%
ToT (ours) ($b=1$)	45%
ToT (ours) ($b=5$)	74%
IO + Refine ($k=10$)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

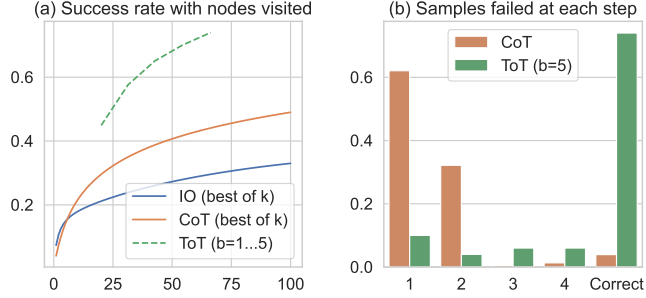


Figure 3: Game of 24 (a) scale analysis & (b) error analysis.

Results. As shown in Table 2, IO, CoT, and CoT-SC prompting methods perform badly on the task, achieving only 7.3%, 4.0%, and 9.0% success rates. In contrast, ToT with a breadth of $b = 1$ already achieves a success rate of 45%, while $b = 5$ achieves 74%. We also consider an oracle setup for IO/CoT, by calculating the success rate using best of k samples ($1 \leq k \leq 100$). To compare IO/CoT (best of k) with ToT, we consider calculating the tree nodes visited per task in ToT across $b = 1 \dots 5$, and map the 5 success rates in Figure 3(a), treating IO/CoT (best of k) as visiting k nodes in a bandit. Not surprisingly, CoT scales better than IO, and best of 100 CoT samples achieve a success rate of 49%, but still much worse than exploring more nodes in ToT ($b > 1$).

Error analysis. Figure 3(b) breaks down at which step CoT and ToT samples fail the task, i.e. the thought (in CoT) or all b thoughts (in ToT) are invalid or impossible to reach 24. Notably, around 60% of CoT samples already failed the task after generating the first step, or equivalently, the first three words (e.g. “4 + 9”). This highlights the issues with direct left-to-right decoding.

4.2 Creative writing

Next, we invent a creative writing task where the input is 4 random sentences and the output should be a coherent passage with 4 paragraphs that end in the 4 input sentences respectively. Such a task is open-ended and exploratory, and challenges creative thinking as well as high-level planning.

Task setup. We sample random sentences from randomwordgenerator.com to form 100 inputs, and there is no groundtruth passage for each input constraint. As we find that GPT-4 can follow the input constraints most of the time, we focus on evaluating passage coherency in two ways: using a GPT-4 zero-shot prompt to provide a 1-10 scalar score, or using human judgments to compare pairs of outputs from different methods. For the former, we sample 5 scores and average them for each task output, and we find these 5 scores usually consistent, with a standard deviation of around 0.56 on average across outputs. For the latter, we employ a subset of the authors in a blind study to compare the coherency of CoT vs. ToT generated passage pairs, where the order of passages is random flipped over 100 inputs.

Baselines. Given the creative nature of the task, both IO and CoT prompts are zero-shot. While the former prompts the LM to directly generate a coherent passage given input constraints, the latter prompts the LM to first make a brief plan then write the passage, i.e. the plan serves as the intermediate thought step. We generate 10 IO and CoT samples per task. We also consider an iterative-refine ($k \leq 5$) method on top of a random IO sample for each task, where the LM is conditioned on input constraints and the last generated passage to decide if the passage is already “perfectly coherent”, and if not generate a refined one.

ToT setup. We build a ToT with depth 2 (and only 1 intermediate thought step) — the LM first generates $k = 5$ plans and votes for the best one (Figure 4), then similarly generate $k = 5$ passages based on the best plan then vote for the best one. Here the breadth limit $b = 1$, as only one choice is kept per step. A simple zero-shot vote prompt (“analyze choices below, then conclude which is most promising for the instruction”) is used to sample 5 votes at both steps.

Results. Figure 5(a) shows average GPT-4 scores across 100 tasks, where ToT (7.56) is deemed to generate more coherent passages than IO (6.19) and CoT (6.93) on average. While such an automatic metric might be noisy, Figure 5(b) confirms the finding by showing that humans prefer ToT over CoT in 41 out of 100 passage pairs, while only prefer CoT over ToT in 21 (other 38 pairs are found “similarly coherent”). Lastly, iterative-refine is more effective on this natural language task, where